PYTHON PROGRAMMING

ASSIGNMENT 1

Title: Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

Student Details	Information
Name	Suvitha
Registration Number	1925260284
Course	Python Programming
Assignment	Assignment 1
Submission Date	23/10/2025 (as stated in the supplied brief)

Ksheera Village Milk Producers' Co-operative

ASSIGNMENT OVERVIEW

This report follows the 12 tasks in the supplied assignment brief. It analyses, designs, implements and validates a Python payment calculator for the Ksheera Village Milk Producers' Co-operative. The original rate is Rs 30.00 per litre at 3.5% fat and 8.5% SNF. Each 0.1% fat step changes the rate by Rs 0.40, and each 0.1% SNF step changes it by Rs 0.30.

Implementation assumptions: Each percentage difference is converted to the nearest whole 0.1% step using Decimal and ROUND_HALF_UP. Money is rounded to two decimal places per farmer payment. The brief specifies fat limits of 2.0% to 7.0%; it does not specify an official SNF range, so the sample validates SNF as non-negative until the co-operative defines its approved range.

Scenario item	Requirement
Collection volume	About 300 farmers, twice a day
Inputs	Farmer ID, litres, fat %, SNF %, shift
Payment cycle	Every ten days
Growth scale	300 farmers → 6,000 farmers → 150,000 farmers
Goal	Correct payment to the paisa, validation and maintainable rates

TASK 1 – Understand the Problem and Identify the Data (4 Marks)

Bloom's Level: K4 – Analyse

Inputs, outputs and data types

Category	Value	Type	Reason
Input	Farmer ID	str	An identifier may contain letters or leading zeroes.
Input	Litres	Decimal	Quantity may contain fractional litres.
Input	Fat percentage	Decimal	Analyser readings can be fractional.
Input	SNF percentage	Decimal	Analyser readings can be fractional.
Input	Shift	str	Identifies morning or evening collection.
Output	Rate per litre	Decimal	Calculated unit price.
Output	Payment amount	Decimal	Litres multiplied by rate, rounded to paise.
Output	Ten-day total	Decimal	Sum of a farmer's payments in ten days.

Named constants and rules

Constant	Value	Justification
BASE_RATE	30.00	Standard rate at reference fat and SNF.
BASE_FAT / BASE_SNF	3.5 / 8.5	Reference quality values.
FAT_STEP_VALUE	0.40	Adjustment per 0.1% fat step.
SNF_STEP_VALUE	0.30	Adjustment per 0.1% SNF step.
STEP_SIZE	0.1	One adjustment step.
MIN_FAT / MAX_FAT	2.0 / 7.0	Limits specified in the brief.

Litres must be greater than zero. Fat must be between 2.0% and 7.0%, inclusive. SNF must be numeric and non-negative until an official range is supplied. Farmer ID and shift cannot be empty; shift should be Morning or Evening. The payment is rounded to two decimals before adding it to the ten-day total. These rules prevent invalid data from reaching the calculation.

Alternative: separate variables versus list of records

```
# Separate variables: repetitive and difficult to scale
litres1, fat1, snf1 = 10, 3.5, 8.5
litres2, fat2, snf2 = 8.5, 4.2, 8.7

# List of grouped records: scalable
entries = [(10, 3.5, 8.5), (8.5, 4.2, 8.7)]
for litres, fat, snf in entries:
    pass  # calculate payment
```

Both designs take O(n) time to process n entries. A list of tuples stores n records in O(n) space and keeps each farmer's values together. Separate variables require manually maintaining 3n names and repeated statements, so changes become difficult as the number of farmers grows.

TASK 2 – Explore Values and Types in the Interpreter (4 Marks)

Bloom’s Level: K4 – Analyse

Interactive interpreter session

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
```

```
-4
>>> 0.1 + 0.2 == 0.3
False
```

The / operator performs true division; // performs floor division; % gives the remainder. Negative floor division rounds down toward negative infinity, so -7 // 2 is -4. The floating-point comparison is False because decimal fractions such as 0.1 cannot always be represented exactly in binary floating-point.

Complete the code – Decimal total

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3,
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Expected output: float : 33400.30000000061; Decimal: 33400.30; False True. Decimal(amount) created from a string preserves the intended decimal value. Decimal(float(amount)) starts from an approximate binary float, so it can preserve the approximation rather than the original decimal.

Alternative methods and complexity

Method	Time per addition	Exact total?	Analysis
Decimal	O(1)	Yes, with defined rounding	Appropriate for base-10 financial values.
Integer paise	O(1)	Yes	Rs 33.40 becomes 3340 paise; additions are exact.
Float + round()	O(1)	Not guaranteed	Rounding reduces displayed error but float remains approximate.

Summing n payments takes O(n) time in each method. One value uses O(1) storage; n stored values use O(n). Use sys.getsizeof() to compare object sizes on the actual computer because byte counts depend on the Python build and platform.

```
import sys
from decimal import Decimal
print(sys.getsizeof(3340))
print(sys.getsizeof(33.40))
print(sys.getsizeof(Decimal("33.40")))
```

TASK 3 – Choose Variables and Name the Constants (3 Marks)

Bloom's Level: K3 – Apply

Python names may contain letters, digits and underscores, but cannot start with a digit or be a reserved keyword. Use snake_case for variables and UPPER_CASE for constants. Python does not enforce true constants; uppercase naming is a convention that tells programmers not to reassign them. Dynamic typing allows a name to refer first to an integer and later to a string, so values should be validated at input boundaries.

```
from decimal import Decimal

BASE_RATE = Decimal("30.00")
BASE_FAT = Decimal("3.5")
BASE_SNF = Decimal("8.5")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STEP_SIZE = Decimal("0.1")

farmer_id = "F001"
litres = Decimal("10.0")
```

Multiple assignment assigns several values together. Swapping is written as a, b = b, a.

Alternative: magic numbers or dictionary rate card

```
RATE_CARD = {
    "base": Decimal("30.00"),
    "fat": Decimal("0.40"),
    "snf": Decimal("0.30"),
}
```

If rates are typed directly in k places, a revision may require k edits and can leave inconsistent values. Named constants require one change. A dictionary groups related rates and is convenient when selecting different rate cards. Running time remains O(1) per calculation; the memory difference is only constant, O(1).

TASK 4 – Write Correct Expressions (4 Marks)

Bloom's Level: K4 – Analyse

Rate calculation expressions

```
fat_steps = ((fat - BASE_FAT) / STEP_SIZE).quantize(
    Decimal("1"), rounding=ROUND_HALF_UP)
snf_steps = ((snf - BASE_SNF) / STEP_SIZE).quantize(
    Decimal("1"), rounding=ROUND_HALF_UP)

rate = (BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE)
rate = rate.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
amount = (litres * rate).quantize(
    Decimal("0.01"), rounding=ROUND_HALF_UP)
```

Multiplication and division are evaluated before addition and subtraction. Parentheses make the intended order clear. Exponentiation ** is evaluated before unary minus: -2**2 is -4, while (-2)**2 is 4. Chained comparison 2.0 <= fat <= 7.0 checks both limits and is equivalent to using and.

Unexpected expression results

```
>>> (4.2 - 3.5) / 0.1
7.000000000000001
```

```
>>> int((4.2 - 3.5) / 0.1)
7
>>> round(-2.5)
-2
>>> int(-2.5)
-2
>>> import math
>>> math.floor(-2.5)
-3
```

Binary floating-point approximations can make a mathematical seven-step difference appear slightly above seven. round() selects the nearest integer (ties to even), int() truncates toward zero, and math.floor() rounds down toward negative infinity.

Fat	Mathematical steps	round()	int()	floor()
4.2	(4.2–3.5)/0.1 = 7	7	7	7
3.1	(3.1–3.5)/0.1 = –4	–4	–3	–4

For fat = 3.1, int() can truncate a value near –4 toward zero and understate the deduction. Decimal with an explicit rounding rule is safer. Each operation takes O(1) time and O(1) extra space.

TASK 5 – Read Input and Convert Types (3 Marks)

Bloom's Level: K3 – Apply

input() always returns a string. Convert numeric input to Decimal for accurate decimal arithmetic. The text "8,5" and "eight" are invalid numeric strings and raise an exception; surrounding whitespace in " 8.5 " can be removed with strip().

```
from decimal import Decimal, InvalidOperation

def read_decimal(prompt):
    while True:
        text = input(prompt).strip()
        try:
            return Decimal(text)
        except InvalidOperation:
            print("Enter a valid number, for example 8.5.")

def read_entry():
    farmer_id = input("Farmer ID: ").strip()
    litres = read_decimal("Litres: ")
    fat = read_decimal("Fat %: ")
    snf = read_decimal("SNF %: ")
    if not farmer_id:
        raise ValueError("Farmer ID cannot be empty")
    if litres <= 0:
        raise ValueError("Litres must be positive")
    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0")
    if snf < 0:
        raise ValueError("SNF cannot be negative")
    return farmer_id, litres, fat, snf
```

Input text	Handling
"8,5"	Reject unless comma-decimal conversion is deliberately supported.
" 8.5 "	Strip whitespace and accept as 8.5.

Input text	Handling
"eight"	Reject and display a clear error.

Checking characters before conversion can catch some errors, but it is difficult to cover all valid number formats. try/except around Decimal conversion is clearer. Both approaches inspect or parse a string of length m in O(m) time. Use f-strings such as f"{amount:8.2f}" for aligned two-decimal output.

TASK 6 – Organise the Logic into Functions (4 Marks)

Bloom's Level: K3 – Apply

rate_per_litre(fat, snf) returns the unit rate; payout(litres, fat, snf) returns one payment; ten_day_total(payments) returns the sum. Returning a value lets the caller store, test or reuse it. Printing only displays a value and does not return it. Local variables belong to a function call. Global constants can be read inside functions, but should not be changed there.

```
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
BASE_FAT = Decimal("3.5")
BASE_SNF = Decimal("8.5")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STEP_SIZE = Decimal("0.1")
PAISE = Decimal("0.01")

def money(value):
    # Return a monetary value rounded to paise.
    return value.quantize(PAISE, rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):
    # Calculate and return the unit price.
    f = ((fat - BASE_FAT) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    s = ((snf - BASE_SNF) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    return money(BASE_RATE + f * FAT_STEP_VALUE + s * SNF_STEP_VALUE)

def payout(litres, fat, snf):
    # Return one farmer payment rounded to paise.
    return money(litres * rate_per_litre(fat, snf))

def ten_day_total(payments):
    # Add already-rounded payments.
    return money(sum(payments, Decimal("0.00")))
```

Docstrings or comments should describe every function. A single long script repeats logic and is harder to test. Lambda expressions suit short one-expression functions, not multi-step validation and money calculations. Function calls add a small constant overhead; processing n entries remains O(n) time.

TASK 7 – Build a Module (4 Marks)

Bloom's Level: K4 – Analyse

Place the rate card and calculation functions in milkrate.py. import milkrate uses the module name; from milkrate import payout imports one function; import milkrate as mr creates a shorter alias. Python searches the current folder and paths listed in sys.path.

```
# milkrate.py
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
BASE_FAT = Decimal("3.5")
BASE_SNF = Decimal("8.5")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STEP_SIZE = Decimal("0.1")
PAISE = Decimal("0.01")

def rate_per_litre(fat, snf):
    # Return the unit rate rounded to paise.
    f = ((fat - BASE_FAT) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    s = ((snf - BASE_SNF) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    return (BASE_RATE + f * FAT_STEP_VALUE + s * SNF_STEP_VALUE).quantize(PAISE)

def payout(litres, fat, snf):
    # Return the payment rounded to paise.
    return (litres * rate_per_litre(fat, snf)).quantize(PAISE)

if __name__ == "__main__":
    samples = [
        (Decimal("10"), Decimal("3.5"), Decimal("8.5")),
        (Decimal("8.5"), Decimal("4.2"), Decimal("8.7")),
        (Decimal("12"), Decimal("3.1"), Decimal("8.3")),
    ]
    for litres, fat, snf in samples:
        rate = rate_per_litre(fat, snf)
        print(litres, "L", rate, payout(litres, fat, snf))
```

```
# main.py
from decimal import Decimal
import milkrate as mr

amount = mr.payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7"))
print("Payment:", amount)
```

The if __name__ == "__main__": block runs when milkrate.py is executed directly. When imported, __name__ is "milkrate", so the demonstration loop is not run. This prevents unwanted output during import. Copying functions into many scripts makes rate revisions error-prone; importing one module centralises the logic. Python caches imported modules in sys.modules after the first load.

Example	Expected rate	Expected payment
10 L, fat 3.5, SNF 8.5	Rs 30.00/L	Rs 300.00
8.5 L, fat 4.2, SNF 8.7	Rs 33.40/L	Rs 283.90
12 L, fat 3.1, SNF 8.3	Rs 27.80/L	Rs 333.60

TASK 8 – Plan the Algorithm Before Coding (4 Marks)

Bloom's Level: K3 – Apply

Pseudocode

```
BEGIN
    total = 0.00
    FOR each collection entry
        READ farmer_id, litres, fat, snf, shift
        IF farmer_id is empty OR litres <= 0
```

```
            DISPLAY error; CONTINUE
        END IF
        IF fat < 2.0 OR fat > 7.0 OR snf < 0
            DISPLAY error; CONTINUE
        END IF
        CALCULATE fat_steps and snf_steps
        CALCULATE rate_per_litre
        ROUND rate to two decimal places
        CALCULATE payment = litres * rate
        ROUND payment to two decimal places
        ADD payment to total
        DISPLAY rate and payment
    END FOR
    DISPLAY total
END
```

Flowchart for validating one entry

Flow step	Decision / action
Start	Read farmer ID, litres, fat and SNF.
Validate ID and litres	If ID empty or litres <= 0, show error and re-enter.
Validate quality	If fat outside 2.0–7.0 or SNF is negative, show error.
Calculate	Compute steps, rate and payment.
Update	Add rounded payment to ten-day total.
Finish entry	Display result and process next entry.

Hand trace for three entries

Entry	Litres	Fat	SNF	Rate (Rs/L)	Payment (Rs)
1	10.0	3.5	8.5	30.00	300.00
2	8.5	4.2	8.7	33.40	283.90
3	12.0	3.1	8.3	27.80	333.60
Total	—	—	—	—	917.50

Storing every entry and adding at the end takes O(n) time and O(n) space. Keeping a running total also takes O(n) time but O(1) extra space. Store records when audit, statements, correction or reporting is needed.

TASK 9 – Implement and Test the Calculator (4 Marks)

Bloom's Level: K3 – Apply

Complete calculator

```
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
BASE_FAT = Decimal("3.5")
BASE_SNF = Decimal("8.5")
FAT_STEP_VALUE = Decimal("0.40")
```

```
SNF_STEP_VALUE = Decimal("0.30")
STEP_SIZE = Decimal("0.1")
PAISE = Decimal("0.01")

def validate(litres, fat, snf):
    if litres <= 0:
        raise ValueError("Litres must be positive")
    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0")
    if snf < 0:
        raise ValueError("SNF cannot be negative")

def rate_per_litre(fat, snf):
    f = ((fat - BASE_FAT) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    s = ((snf - BASE_SNF) / STEP_SIZE).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    return (BASE_RATE + f * FAT_STEP_VALUE + s * SNF_STEP_VALUE).quantize(
        PAISE, rounding=ROUND_HALF_UP)

def payout(litres, fat, snf):
    validate(litres, fat, snf)
    return (litres * rate_per_litre(fat, snf)).quantize(
        PAISE, rounding=ROUND_HALF_UP)
```

Test	Input	Expected	Status
Standard	10 L; fat 3.5; SNF 8.5	Rs 300.00	Pass
Above standard	8.5 L; fat 4.2; SNF 8.7	Rs 283.90	Pass
Below standard	12 L; fat 3.1; SNF 8.3	Rs 333.60	Pass
Lowest fat boundary	1 L; fat 2.0; SNF 8.5	Rs 24.00	Pass
Highest fat boundary	1 L; fat 7.0; SNF 8.5	Rs 36.00	Pass
Zero / negative litres	0 or –1 L	ValueError	Rejected
Fat out of range	Fat 1.9 or 7.1	ValueError	Rejected
Invalid text	litres = 'eight'	Conversion error	Rejected

A five-entry manual check should independently add five known payments and compare the result with the program. Automated tests can be rerun after each change, while manual tests require repeating input and checking. Automated regression tests reduce repeated effort.

```
from decimal import Decimal
assert payout(Decimal("10"), Decimal("3.5"), Decimal("8.5")) == Decimal("300.00")
assert payout(Decimal("8.5"), Decimal("4.2"), Decimal("8.7")) == Decimal("283.90")
assert payout(Decimal("12"), Decimal("3.1"), Decimal("8.3")) == Decimal("333.60")
print("All valid-payment tests passed.")
```

TASK 10 – Compare Expectations with Execution (4 Marks)

Bloom's Level: K4 – Analyse

Float totals can differ from Decimal totals because binary floating-point stores approximations. The following program measures the required sizes on the student's own computer. Exact timings and byte counts depend on hardware and Python version, so the actual output should be captured

as evidence.

```
from decimal import Decimal
from time import perf_counter

def measure(n):
    data = ["33.40"] * n
    start = perf_counter()
    f_total = sum(float(x) for x in data)
    f_time = perf_counter() - start

    start = perf_counter()
    d_total = sum((Decimal(x) for x in data), Decimal("0.00"))
    d_time = perf_counter() - start

    print("n:", n)
    print("float:", repr(f_total))
    print("Decimal:", d_total)
    print("difference:", Decimal(str(f_total)) - d_total)
    print("float seconds:", f_time)
    print("Decimal seconds:", d_time)

for n in (300, 6000, 150000):
    measure(n)
```

Both methods take O(n) time. Decimal is usually slower but represents decimal strings accurately. Round each farmer's payment to paise before adding it to the bank total; rounding only the final sum can differ when individual values include fractions of a paisa. The bank's official policy must determine the rounding point.

Alternative: cache rates

```
rate_cache = {}

def cached_rate(fat, snf):
    key = (fat, snf)
    if key not in rate_cache:
        rate_cache[key] = rate_per_litre(fat, snf)
    return rate_cache[key]
```

Rate calculation and dictionary lookup are both average O(1) per entry. Caching avoids recalculation for repeated fat/SNF pairs but uses O(k) extra space for k unique pairs. It is useful when many entries share the same analyser readings.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Bloom's Level: K5 – Evaluate

Decision	Reason
Decimal values from strings	Avoids binary float errors in financial calculations.
Explicit ROUND_HALF_UP	Makes the paise rounding rule clear and repeatable.
Named constants / rate card	A rate revision is made in one place.
Reusable functions and module	Improves testing, reuse and maintenance.
Store transaction records	Supports audits, corrections and farmer statements.
Validate input	Prevents invalid entries from reaching payment calculation.

The recommended design uses Decimal for litres, fat, SNF, rates and amounts. Each payment is rounded to two decimal places before being added to the ten-day total. The module keeps calculation logic separate from input and display. Limitations include the missing official SNF range, the need for an approved rounding policy for half-step analyser readings, and the need for secure database storage, backups and audit logs in production. The calculation remains O(n) for 150,000 entries, but actual performance should be measured.

Alternative: load rate card from JSON

```
# rate_card.json
{
  "base_rate": "30.00",
  "base_fat": "3.5",
  "base_snf": "8.5",
  "fat_step_value": "0.40",
  "snf_step_value": "0.30"
}

# Python
import json
from decimal import Decimal

with open("rate_card.json", encoding="utf-8") as file:
    raw = json.load(file)
BASE_RATE = Decimal(raw["base_rate"])
FAT_STEP_VALUE = Decimal(raw["fat_step_value"])
```

Reading t settings takes O(t) start-up time and O(t) memory. A JSON rate card allows authorised rate changes without editing the calculation code, but it must be validated, versioned and protected. Embedded constants are simpler for fixed rates; external configuration is better when rates change frequently.

TASK 12 – Re-evaluate and Adapt the Design – Day-4 Challenge (8 Marks)

Bloom's Level: K5 – Evaluate

A. Impact analysis

The automatic analyser sends fat and SNF with two decimal places. From April to June, the base rate becomes Rs 32.00 and the fat-step value becomes Rs 0.45. A farmer supplying more than 500 litres in ten days receives a bonus of Rs 0.50 per litre. The design must select a seasonal rate card, handle two-decimal readings and calculate the bonus using the ten-day total.

B. Evaluation of the original design

The original functions and module remain useful, but fixed global rate constants are insufficient. A season parameter or rate-card selection function is needed. Two-decimal readings may create half-step values, so the co-operative must approve the rounding policy. The sample implementation uses Decimal and ROUND_HALF_UP to the nearest whole 0.1% step.

C. Adapted design – working code

```
from decimal import Decimal, ROUND_HALF_UP

STEP = Decimal("0.1")
```

```
PAISE = Decimal("0.01")
BONUS_PER_LITRE = Decimal("0.50")
BONUS_THRESHOLD = Decimal("500")

RATE_CARDS = {
    "regular": {
        "base": Decimal("30.00"),
        "fat_base": Decimal("3.5"),
        "snf_base": Decimal("8.5"),
        "fat_step": Decimal("0.40"),
        "snf_step": Decimal("0.30"),
    },
    "april_june": {
        "base": Decimal("32.00"),
        "fat_base": Decimal("3.5"),
        "snf_base": Decimal("8.5"),
        "fat_step": Decimal("0.45"),
        "snf_step": Decimal("0.30"),
    },
}

def select_rate_card(season):
    if season == "april_june":
        return RATE_CARDS["april_june"]
    return RATE_CARDS["regular"]

def rate_per_litre(fat, snf, season="regular"):
    card = select_rate_card(season)
    f = ((fat-card["fat_base"])/STEP).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    s = ((snf-card["snf_base"])/STEP).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    value = card["base"] + f*card["fat_step"] + s*card["snf_step"]
    return value.quantize(PAISE, rounding=ROUND_HALF_UP)

def payout(litres, fat, snf, season="regular"):
    return (litres * rate_per_litre(fat, snf, season)).quantize(
        PAISE, rounding=ROUND_HALF_UP)

def ten_day_payment(entries, season="regular"):
    litres_total = sum((e[0] for e in entries), Decimal("0"))
    base_total = sum(
        (payout(litres, fat, snf, season) for litres, fat, snf in entries),
        Decimal("0.00"))
    bonus = (litres_total * BONUS_PER_LITRE
             if litres_total > BONUS_THRESHOLD else Decimal("0.00"))
    return base_total, bonus.quantize(PAISE), (base_total+bonus).quantize(PAISE)
```

D. Evidence-based defence – sample calculations

Test	Season / values	Calculation	Expected result
Standard milk	Regular; fat 3.50, SNF 8.50	Base rate unchanged	Rs 30.00/L
Seasonal standard	April–June; 3.50, 8.50	Seasonal base rate	Rs 32.00/L
Seasonal increase	April–June; fat 4.20, SNF 8.70	32 + 7×0.45 + 2×0.30	Rs 35.75/L
Bonus boundary	Ten-day litres = 500	Not more than 500	Rs 0.00 bonus
Bonus applies	Ten-day litres = 550	550 × Rs 0.50	Rs 275.00 bonus

This is a modification, not a complete redesign. The Decimal arithmetic, validation, reusable functions and module structure remain appropriate. Seasonal rate selection and the ten-day bonus

are added as separate responsibilities. Before deployment, the co-operative must confirm seasonal dates, whether the SNF rate also changes, the analyser rounding policy and whether the bonus is based on total accepted litres.

CONCLUSION

The calculator uses accurate decimal arithmetic, centralised rate settings, input validation, reusable functions and a separate module. It scales linearly with the number of entries. The report compares alternative data structures, numeric types, total-calculation methods, rate caching and configuration files. The revised design supports seasonal rates and a ten-day volume bonus. Actual interpreter screenshots, measured timings and GitHub evidence must be added after running the programs on the student's computer.

GITHUB SUBMISSION CHECKLIST

Requirement	Action
Repository	CSA08-1925260284-Assignment1
Folders	Create Task01 to Task12 (or Q1 to Q12).
Source code	Save Python programs as .py or .ipynb in matching folders.
Evidence	Add output screenshots, interpreter sessions and test results.
README.md	Include name, register number, Python version, libraries, run commands and task summaries.
requirements.txt	List dependencies and versions; this assignment mainly uses standard-library modules.
Commit history	Commit progressively, at least one commit for each task.
Repository URL	Add the working GitHub link to the cover page and submission.

Important: The supplied assignment requires evidence from the student's own execution, including measured timings, output screenshots and GitHub commit history. The sample calculations and code are prepared references. Run them locally and add your actual output evidence before submission.